StockFlow

Smart Inventory & Procurement Platform

User guide, system overview, technology reference, and operating instructions

StockFlow

Smart Inventory & Procurement Platform

StockFlow is a business operations website for organizations that manage products across multiple warehouses and regularly purchase stock from suppliers.

It brings inventory, procurement, supplier coordination, approvals, demand planning, alerts, and operational history into one system.

The goal is simple: help a business know what it has, where it is, what it will need, and when it should purchase more.

1. Why StockFlow is needed

Many businesses manage inventory using spreadsheets, chat messages, emails, and disconnected systems. That creates common problems:

- Staff do not know the real stock level in each warehouse.
- Reserved stock is confused with available stock.
- Products reach zero before a replenishment order is approved.
- Purchase orders are created without a clear approval process.
- Supplier lead times are difficult to track.
- Goods received are not always added to inventory immediately.
- Warehouse transfers are difficult to audit.
- Managers cannot easily see who changed inventory or approved an order.
- Demand planning is based on guesswork instead of recorded demand.

StockFlow addresses these problems by creating one operational source of truth.

It helps teams:

1. Track stock by warehouse.
2. Separate on-hand, reserved, and available quantities.
3. Identify low-stock products before they become unavailable.
4. Create and approve purchase orders.
5. Record the exact products and quantities ordered.
6. Receive goods and automatically increase stock.
7. Transfer stock between warehouses.
8. Review inventory movement history.
9. Record suppliers and lead times.
10. Forecast future demand.
11. Control access using user roles.
12. Maintain an audit trail of important actions.

2. What the website does

Overview dashboard

The Overview page is the operations center.

It shows:

- Total inventory value
- Total units currently on hand
- Number of active products
- Number of low-stock alerts
- Products that need replenishment
- Warehouses where low stock is located
- Purchase orders waiting for approval
- Quick links to inventory, procurement, and forecasting

This page is designed for a manager who wants to understand the current situation quickly.

Inventory

The Inventory page shows stock by product and warehouse.

Each inventory record includes:

- SKU
- Product name
- Warehouse
- Quantity on hand
- Reserved quantity
- Available quantity
- Reorder point
- Stock health status

Available stock is calculated as:

```
Available stock = On-hand quantity - Reserved quantity
```

Users with the correct role can update stock quantities. The system prevents invalid values such as negative stock or reserved stock greater than the total quantity.

Products and categories

Products represent the items the business buys, stores, and sells.

Each product can contain:

- SKU
- Product name
- Category
- Reorder point
- Unit cost

Categories organize products into groups such as Hardware, Consumables, Packaging, or Office Supplies.

Products and categories can be created, edited, and deleted according to user permissions. A product with existing inventory cannot be deleted accidentally.

Warehouses

Warehouses represent physical storage locations such as distribution centers, stores, regional hubs, or fulfillment locations.

Each warehouse includes:

- Name
- Location
- Total units stored there

The same product can exist in multiple warehouses with different stock levels.

Suppliers

The Suppliers page stores purchasing information.

Each supplier can include:

- Supplier name
- Email address
- Expected lead time in days
- Number of purchase orders

Lead time is useful for deciding when a replenishment order should be created.

Purchase orders

Purchase orders represent planned purchases from suppliers.

A purchase order includes:

- Supplier
- Destination warehouse
- Status
- Total value
- Creation date
- Product line items
- Ordered quantity
- Unit cost
- Received quantity

Typical purchase-order statuses are:

- `PENDING_APPROVAL` — created but waiting for approval
- `APPROVED` — approved for purchasing
- `REJECTED` — rejected by an authorized manager
- `RECEIVED` — all ordered items have been received

The platform supports a basic approval workflow so procurement does not happen without review.

Goods receiving

When goods arrive from a supplier, an authorized user records the received quantity against a purchase-order line.

StockFlow then:

1. Validates that the received quantity does not exceed the ordered quantity.
2. Adds the received quantity to the destination warehouse.
3. Updates the purchase-order line received quantity.
4. Records an inventory transaction.
5. Marks the purchase order as received when all lines are complete.
6. Adds an audit-log entry.

This prevents received stock from being forgotten or added manually in a separate spreadsheet.

Warehouse transfers

Warehouse transfers move stock from one location to another.

Before completing a transfer, StockFlow checks that:

- The source and destination warehouses are different.
- The requested quantity is positive.
- The source warehouse has enough available stock.

The system then:

- Decreases stock in the source warehouse.
- Increases stock in the destination warehouse.
- Creates transfer-out and transfer-in transactions.
- Creates a transfer record.
- Records the action in the audit log.

Low-stock alerts

An item becomes low-stock when its available quantity is less than or equal to its reorder point.

StockFlow supports:

- Low-stock indicators on the dashboard
- In-app alert records
- Alert generation from current inventory
- Alert resolution by authorized users
- Optional email notifications through SMTP

Email is optional. The website continues to work with in-app alerts when SMTP is not configured.

Demand forecasting

The Forecast page estimates future demand for a selected product.

The current forecasting foundation uses a transparent trend-adjusted moving-average approach. It uses recorded demand events when available and falls back to a baseline based on the product reorder point for new products.

The forecast returns:

- Forecast method
- Number of historical demand points
- Estimated daily demand
- Forecast units for the selected horizon
- Recommended order quantity

This approach is intentionally explainable and lightweight. It can later be replaced or extended with a trained machine-learning model using historical sales, seasonality, promotions, and supplier lead times.

Inventory transaction history

StockFlow records important inventory movements such as:

- Goods receipts
- Warehouse transfers in
- Warehouse transfers out
- Demand adjustments

This history helps answer questions such as:

- Why did the stock level change?
- When was the stock received?
- Which warehouse sent the stock?
- Which warehouse received it?
- What reference or purchase order caused the change?

Audit logs

Audit logs record important user actions, including:

- Product creation, updates, and deletion
- Category creation, updates, and deletion
- Purchase-order line additions
- Goods receiving
- Warehouse transfers
- Alert resolution

Managers and administrators can review these logs for accountability and troubleshooting.

3. User roles and permissions

StockFlow uses role-based access control.

Admin

The Admin role has the highest access level.

Admins can:

- Manage users

- Manage products and categories
- Manage warehouses and suppliers
- Create and approve purchase orders
- Receive goods
- Transfer inventory
- Resolve alerts
- Review audit logs

Manager

Managers can operate inventory and procurement workflows.

They can:

- Update stock
- Approve or reject purchase orders
- Receive goods
- Transfer stock
- Manage products and categories
- Resolve alerts
- Review audit logs

Procurement

Procurement users focus on purchasing.

They can:

- Create purchase orders
- Add purchase-order line items
- Manage suppliers
- View products, warehouses, and inventory

Viewer

Viewers have read-only access to operational information.

They can view:

- Dashboard metrics
- Inventory
- Warehouses
- Products
- Suppliers
- Purchase orders
- Forecasts

They cannot change protected business data.

4. Technology used

React

React is used for the website interface.

Why React is used:

- It supports reusable interface components.
- It updates screens without reloading the entire page.
- It is suitable for dashboards, forms, tables, and workflow screens.
- It makes it easier to add future pages such as reports and administration.

Vite

Vite is used to run and build the frontend.

Why Vite is used:

- Very fast development server.
- Fast production builds.
- Simple configuration for a React application.

FastAPI

FastAPI provides the backend REST API.

Why FastAPI is used:

- High performance and asynchronous support.
- Automatic API documentation.
- Clear request validation using Pydantic.
- Easy integration with React.
- Built-in support for dependency injection and authentication patterns.

The API documentation is available at:

```
http://localhost:8000/docs
```

Python

Python powers the backend business logic and forecasting service.

Why Python is used:

- Strong ecosystem for data processing and forecasting.
- Clear syntax for business rules.
- Easy integration with machine-learning tools later.
- Good support for testing and automation.

SQLAlchemy

SQLAlchemy is used as the database access layer.

Why SQLAlchemy is used:

- Maps Python models to database tables.

- Supports SQLite for local development.
- Supports PostgreSQL for production.
- Helps keep database queries structured and maintainable.

PostgreSQL

PostgreSQL is the recommended production database.

Why PostgreSQL is used:

- Reliable relational data storage.
- Strong transaction support.
- Suitable for multi-user business applications.
- Good support for constraints, indexes, reporting, and future scaling.

SQLite is used by default for local development so the project can be started without installing a database server.

Alembic

Alembic manages database migrations.

Why Alembic is used:

- Tracks database schema changes.
- Makes upgrades repeatable.
- Helps keep development, test, and production databases consistent.
- Runs automatically in the Docker API container before the server starts.

Docker and Docker Compose

Docker packages the application and its dependencies into repeatable services.

The Compose setup contains:

- `db` — PostgreSQL database
- `api` — FastAPI backend
- `web` — production frontend served by Nginx

Why Docker is used:

- The same environment can be used by developers and deployment servers.
- PostgreSQL setup is automated.
- Dependencies are isolated.
- The application can be started with one command.

Pytest

Pytest is used for automated backend tests.

The tests cover:

- Health checks
- Demo data and dashboard metrics
- Authentication

- Protected category operations
- Warehouse transfers
- Alerts
- Purchase-order lines
- Goods receiving
- Security headers

5. Project structure

```
stockflow/
■■■ backend/
■ ■■■ app/
■ ■ ■■■ main.py Backend API and business logic
■ ■■■ migrations/ Alembic migration configuration
■ ■■■ tests/ Automated backend tests
■ ■■■ requirements.txt Python dependencies
■ ■■■ Dockerfile Backend container image
■ ■■■ .env.example Backend environment template
■■■ frontend/
■ ■■■ src/
■ ■ ■■■ main.jsx Main React application
■ ■ ■■■ styles.css Website styling
■ ■ ■■■ platform.jsx Authentication and administration area
■ ■ ■■■ products.jsx Product CRUD screen
■ ■ ■■■ categories.jsx Category CRUD screen
■ ■ ■■■ operations.jsx Transfers, alerts, and history
■ ■ ■■■ procurement.jsx PO lines and receiving
■ ■■■ package.json Frontend dependencies and scripts
■ ■■■ Dockerfile Frontend container image
■ ■■■ .env.example Frontend environment template
■■■ docker-compose.yml Full local deployment definition
■■■ README.md This guide
```

6. Local development without Docker

Requirements

- Windows PowerShell
- Python 3.12 or later
- Node.js and npm

Start the backend

Open PowerShell and run:

```
cd "C:\Users\RAM\Desktop\Mycodex stuff\stockflow\backend"
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
uvicorn app.main:app --reload
```

The backend will run at:

```
http://localhost:8000
```

Useful backend URLs:

- Health check: http://localhost:8000/health

- API documentation: http://localhost:8000/docs

The root URL `/` is not the website homepage, so it may return `{"detail":"Not Found"}`. That is expected.

Start the frontend

Open a second PowerShell window and run:

```
cd "C:\Users\RAM\Desktop\Mycodex stuff\stockflow\frontend"
npm install
npm run dev
```

Open the URL shown by Vite, normally:

```
http://localhost:5173
```

Keep both terminals open while using the website.

7. Run with Docker

Make sure Docker Desktop is installed and running.

From the project root:

```
cd "C:\Users\RAM\Desktop\Mycodex stuff\stockflow"
docker compose up --build
```

Open:

```
http://localhost:5173
```

The service ports are:

- Frontend: `5173`
- API: `8000`
- PostgreSQL: internal Docker network port `5432`

To stop the services:

```
docker compose down
```

To stop the services and remove the database volume:

```
docker compose down -v
```

Removing the volume deletes the Docker PostgreSQL data, so use that command only when you intentionally want a fresh database.

Docker troubleshooting

If you see:

```
failed to connect to the docker API
```

Docker Desktop is not running. Start Docker Desktop, wait for the Docker Engine to become ready, and run the Compose command again.

If the frontend shows `Failed to fetch`, the API container is not reachable. Check the API logs:

```
docker compose logs api
```

If the API shows database connection errors, check the database logs:

```
docker compose logs db
```

8. First-time website setup

1. Start the backend and frontend, or start Docker Compose.
2. Open http://localhost:5173.
3. Select the **Platform** area.
4. Click **Create first admin**.
5. Enter an email address and a password with at least eight characters.
6. Sign in.
7. Return to the Overview page.
8. Click **Seed demo data** to create sample warehouses, products, suppliers, and inventory.

The first account becomes an Admin. After an Admin exists, additional users must be created by an authorized administrator.

9. Typical daily workflow

Morning inventory review

1. Open Overview.
2. Check inventory value and total units.
3. Review low-stock alerts.
4. Open Inventory to confirm the warehouse and available quantity.
5. Open Forecast for products with recurring demand.

Replenishment workflow

1. Review a low-stock product.
2. Run its forecast.
3. Check supplier lead time.
4. Create a purchase order for the selected supplier and warehouse.
5. Add product line items, quantities, and unit costs.
6. Submit the order for approval.
7. A Manager or Admin approves or rejects it.

Receiving workflow

1. Open the procurement receiving area.
2. Select the purchase order.
3. Select the purchase-order line.
4. Enter the quantity received.
5. Confirm receipt.
6. StockFlow updates inventory and creates a receipt transaction.

Transfer workflow

1. Open the inventory workflows area.
2. Select the product.
3. Select the source warehouse.
4. Select the destination warehouse.
5. Enter the quantity.
6. Complete the transfer.
7. Review the resulting transfer transactions.

Alert workflow

1. Generate alerts from current inventory.
2. Review unresolved alerts.
3. Investigate the product and warehouse.
4. Create a purchase order if replenishment is needed.
5. Resolve the alert after the issue is handled.

10. Database configuration

Local SQLite

If `DATABASE_URL` is not set, the API uses a local SQLite database named `stockflow.db` inside the backend directory.

This is convenient for learning and local testing.

PostgreSQL

Set the database URL before starting the backend:

```
$env:DATABASE_URL = "postgresql+psycopg://stockflow:password@localhost:5432/stockflow"
$env:STOCKFLOW_AUTH_SECRET = "replace-with-a-long-random-secret"
uvicorn app.main:app --reload
```

For production, use a secret manager instead of committing credentials to files.

11. Environment variables

Backend

See `backend/.env.example`.

- `DATABASE_URL` — database connection string
- `STOCKFLOW_AUTH_SECRET` — token signing secret
- `STOCKFLOW_CORS_ORIGINS` — comma-separated allowed frontend origins
- `STOCKFLOW_AUTO_CREATE_TABLES` — local table creation switch
- `SMTP_HOST` — SMTP server hostname
- `SMTP_PORT` — SMTP server port
- `SMTP_FROM` — sender address

- `SMTP_USER` — SMTP username
- `SMTP_PASSWORD` — SMTP password
- `ALERT_RECIPIENT` — alert destination email

Frontend

See `frontend/.env.example`.

- `VITE_API_URL` — URL of the FastAPI backend

12. Database migrations

The project includes Alembic migrations.

From the backend directory:

```
alembic upgrade head
```

To create a new migration after changing database models:

```
alembic revision --autogenerate -m "describe the change"
alembic upgrade head
```

The Docker API container runs migrations before starting the API server.

13. Automated tests

From the project root:

```
cd backend
.\.venv\Scripts\Activate.ps1
pytest -q
```

The test suite validates the core API and workflow behavior without requiring the frontend browser.

The frontend production build can be checked with:

```
cd frontend
npm run build
```

14. API overview

Important endpoints include:

- `GET /health` — service health
- `POST /api/auth/register` — create the first Admin or an Admin-created user
- `POST /api/auth/login` — sign in
- `GET /api/auth/me` — inspect the current user
- `GET /api/dashboard` — dashboard metrics
- `GET /api/products` — list products
- `POST /api/products` — create a product
- `PATCH /api/products/{id}` — update a product
- `DELETE /api/products/{id}` — delete a product
- `GET /api/categories` — list categories

- `POST /api/categories` — create a category
- `PATCH /api/categories/{id}` — update a category
- `DELETE /api/categories/{id}` — delete a category
- `GET /api/inventory` — inventory by warehouse
- `POST /api/stock` — update stock
- `GET /api/warehouses` — list warehouses
- `GET /api/suppliers` — list suppliers
- `POST /api/suppliers` — create a supplier
- `GET /api/purchase-orders` — list purchase orders
- `POST /api/purchase-orders` — create a purchase order
- `POST /api/purchase-orders/{id}/items` — add a purchase-order line
- `POST /api/purchase-orders/{id}/receive` — receive goods
- `PATCH /api/purchase-orders/{id}/approve` — approve an order
- `PATCH /api/purchase-orders/{id}/reject` — reject an order
- `POST /api/transfers` — transfer stock
- `GET /api/inventory/transactions` — inventory history
- `GET /api/alerts` — unresolved alerts
- `POST /api/alerts/generate` — generate alerts
- `PATCH /api/alerts/{id}/resolve` — resolve an alert
- `GET /api/forecast/{product_id}` — run a forecast
- `GET /api/audit-logs` — protected audit history

FastAPI automatically exposes interactive documentation at `/docs` and a machine-readable OpenAPI document at `/openapi.json`.

15. Security and production notes

StockFlow includes several baseline security controls:

- Passwords are stored using PBKDF2 hashing with per-password salts.
- Authentication tokens are signed and expire.
- Protected operations check user roles.
- Production startup rejects the default authentication secret.
- CORS origins are configurable instead of allowing every origin.
- Security response headers are added by the API.
- Inventory and receiving operations validate quantities.
- Audit logs record important mutations.
- Database credentials and SMTP credentials are configured through environment variables.

Before public deployment, also configure:

- HTTPS/TLS

- A strong secret manager
- A production PostgreSQL password
- A real SMTP provider if email alerts are required
- Database backups
- Centralized logs and monitoring
- A reverse proxy or managed ingress
- Regular dependency updates
- A production identity provider if enterprise SSO is required

16. Future extensions

The platform is structured to support future enhancements such as:

- Advanced machine-learning forecasting with seasonal models
- Sales and ERP integrations
- Barcode scanning
- Batch and lot tracking
- Expiry-date management
- Multi-currency purchasing
- Approval thresholds by spend amount
- Scheduled background forecast jobs
- Mobile warehouse receiving
- Supplier scorecards
- Advanced analytics and exportable reports

Summary

StockFlow is designed to make inventory and purchasing decisions visible, controlled, and auditable.

The website connects warehouse stock, suppliers, purchase orders, receiving, transfers, alerts, forecasting, and approvals in one operational workflow. React provides the user interface, FastAPI provides the business API, Python supports the forecasting logic, SQLAlchemy connects the application to SQLite or PostgreSQL, Alembic manages schema changes, and Docker packages the complete system for repeatable deployment.